

1. Project Title: Bookshelf

2. Report:

- Feature(pt 5) + Prototype = Ritikha Ashok – the system shall allow the user to import books from the device's Download folder and support both .txt and .pdf formats.
- Comparison(pt 6) = Sarah
- Conclusion(pt 7) = Bo
- Test plan(pt 5) = Krish
- Project scheduling(pt 4) = Bhumish
- Costs estimation(pt 4) = Tri

Github: <https://github.com/ritikhaashok/3354-Team9.git>

2. Group Members and assigned sections

- Bo Seong Kim
 - Project scope (1.5)
 - Functional Requirements
- Bhumish Dayal
 - Architectural Design(MVC Pattern)
- Krish Koirala
 - Sequence Diagram
- Sarah Parratt
 - Use Case Diagram
- Tri Nguyen
 - Github Setup (1.4)
 - Non-functional requirements
- Ritikha Ashok
 - Github Setup (1.3)
 - Class Diagram

3. Assumptions: None

4. Feedback provided from Final Project Proposal: “Well done.”

5. Software process model: Prototyping - Because our app's purpose is very user-heavy, it is easiest to build rapid prototypes and frequently test feature designs with users to gather feedback before implementing them.

6a. Functional requirements - Bo Seong Kim:

1. The system shall allow the user to import books from the device's Download folder.

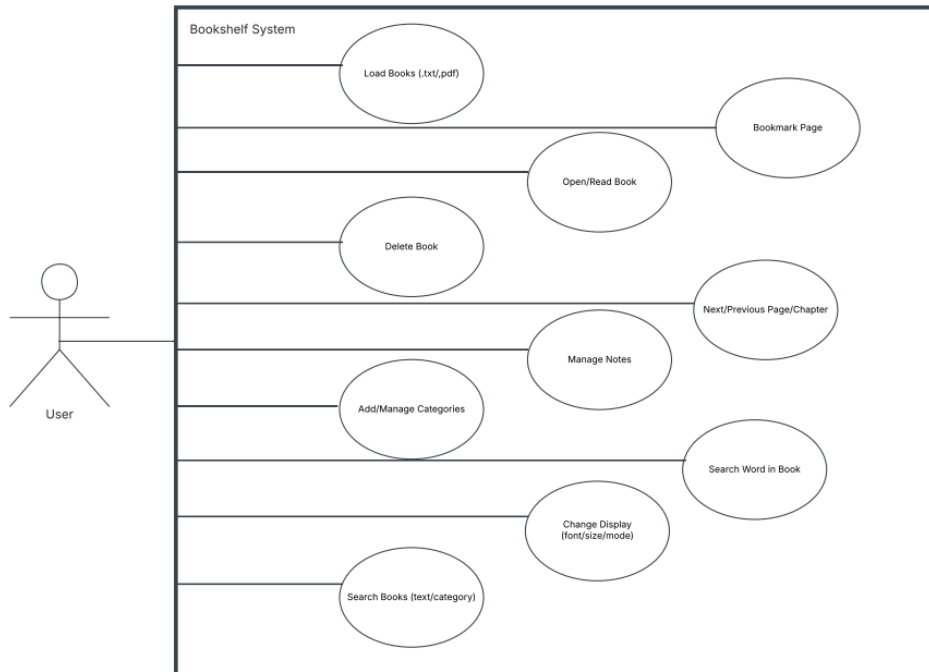
2. The system shall add the imported books if it succeeds in importing.
3. The system shall allow the user to delete books from the library.
4. The system shall allow the user to search for books using a text query.
5. The system shall display the content of the book that the user requested.
6. The system shall allow the user to navigate between pages.
7. The system shall allow the user to create bookmarks in the reader.

6b. Non-functional requirements - Tri Nguyen:

1. Usability: The system shall have a clear and intuitive navigation menu
2. Performance: The system shall take less than 7 seconds for basic operations like search and categorization
3. Space: The system shall allow the user to back up 5 GB of books and data to the cloud
4. Dependability: The system shall have downtime of less than 10 seconds on a working day
5. Security: The system shall allow users to authenticate themselves using an account and 2FA for accessing and making backups
6. Environmental: The system shall support all iOS and Android mobile devices
7. Operational: The system shall always be available except for scheduled maintenance
8. Development: The system shall be developed using Java
9. Regulatory: The system shall not allow restricted or banned books
10. Ethical: The system shall keep user information confidential
11. Accounting: The system shall maintain a log of account creation/modification activities
12. Safety/security: The system shall store user information securely

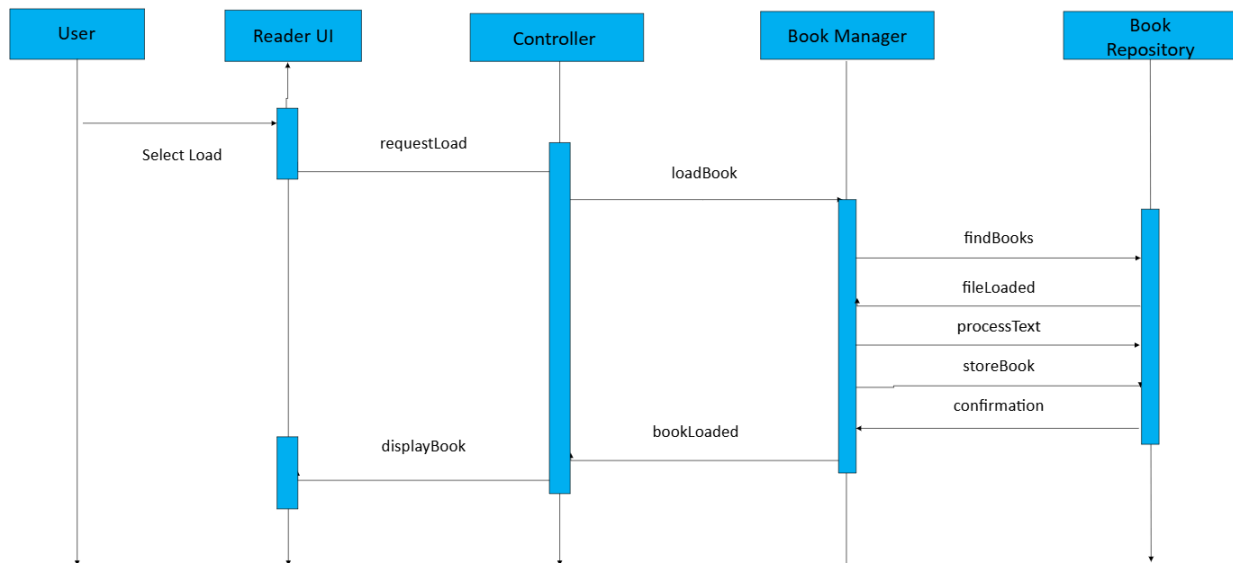
7. Use Case Diagram - Sarah Paratt:

Bookshelf Use Case Diagram

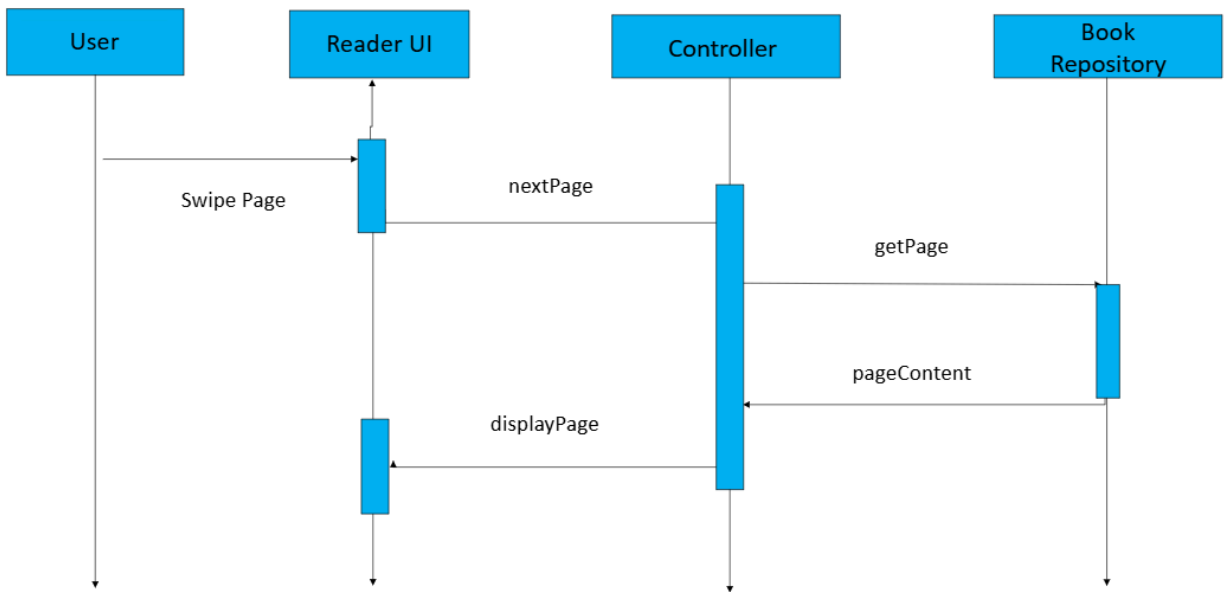


8. Sequence Diagrams - Krish

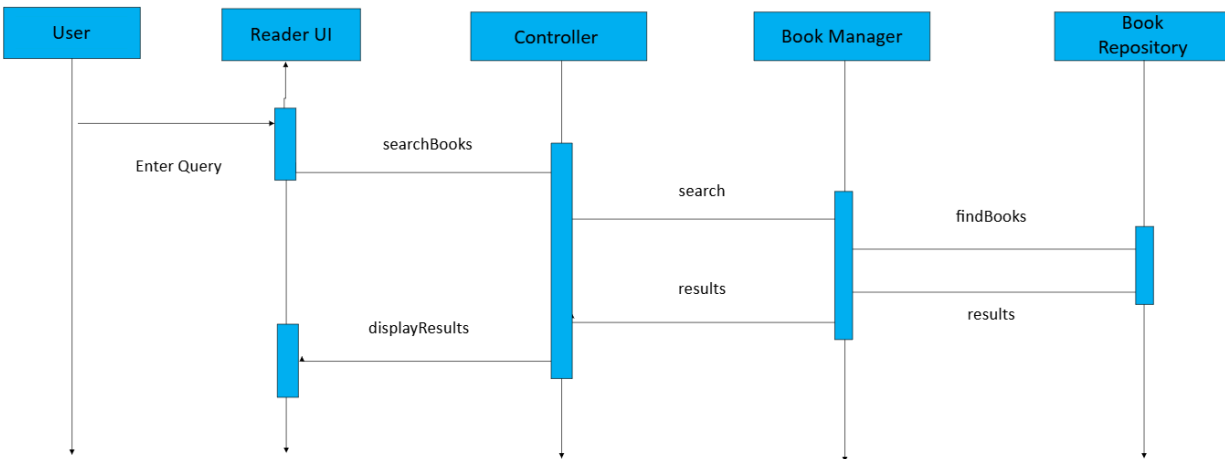
Load Book Sequence:



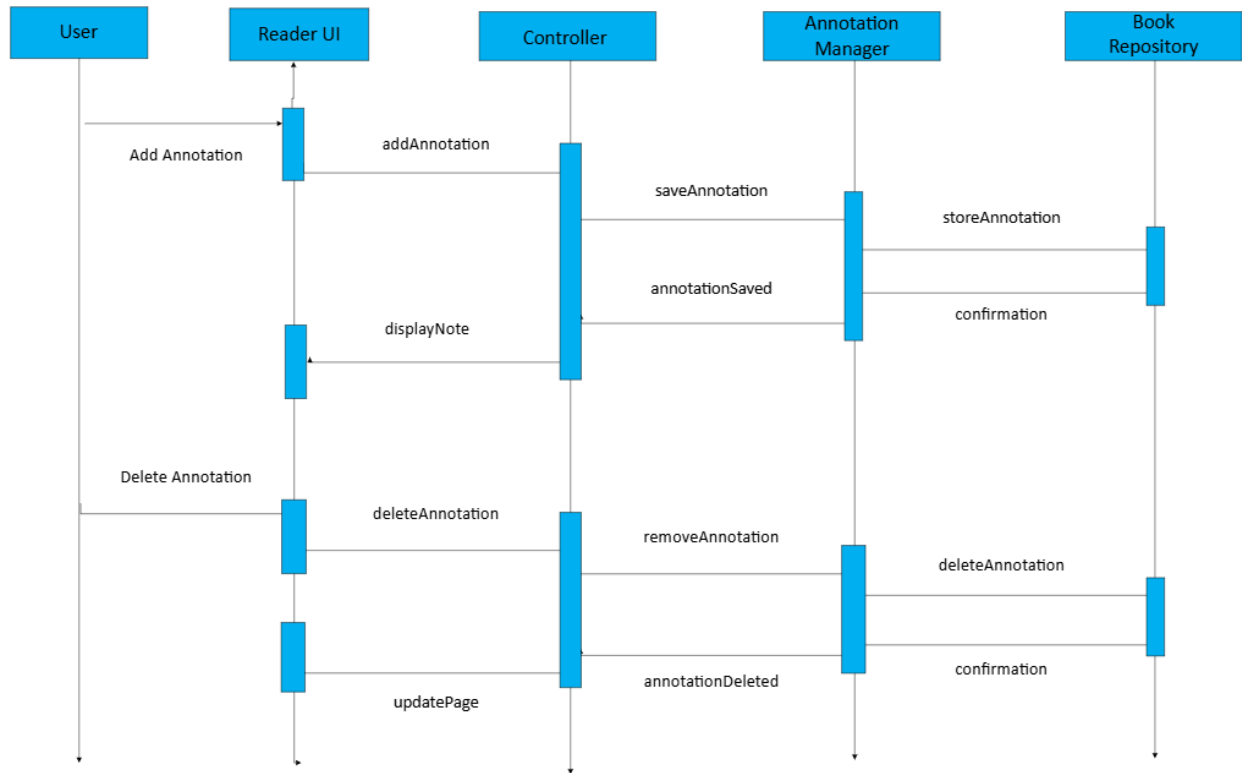
Read Book Sequence:



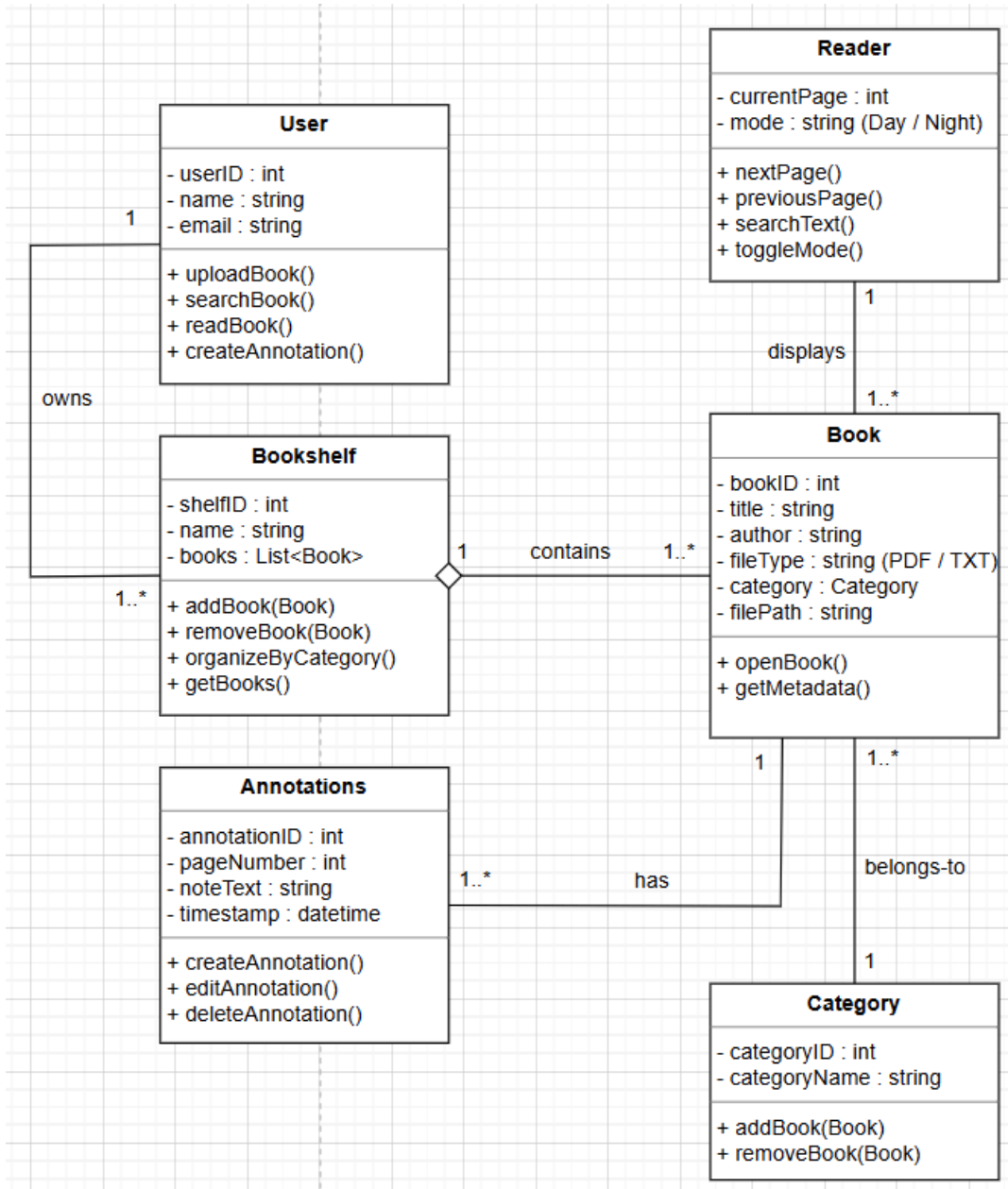
Search Books Sequence:



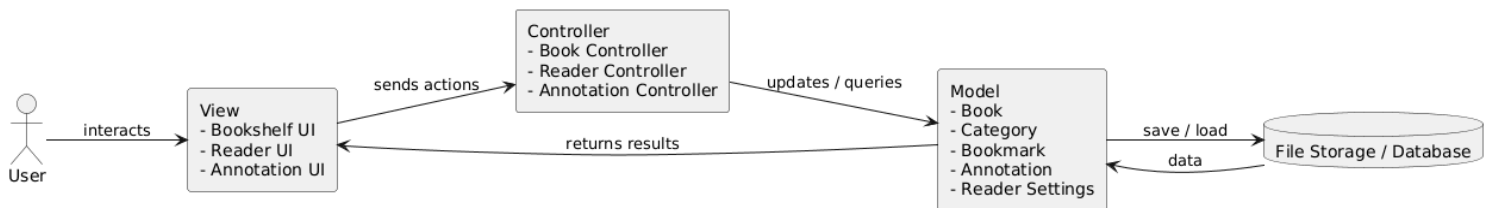
Annotation Sequence:



9. Class Diagram - Ritikha



10. MVC Model - Bhumish



Project Scheduling: Make an estimation of the schedule of your project.

i. Please provide a start date, and end date by giving justifications about your estimation.

The estimated start date for our project was between the last week of January and the first week of February and the estimated end date is April 5th, 2026. The schedule was dependent on how and when the deliverables were due. Part one was mainly an introduction to the team and all that, and the actual project work began at the start of February, before the submission for part two that was on February 8, 2026. After that, the team continued working on the project for Part 3 that was due on March 8, 2026, and is still working on it for Part 4 report and the presentation, which is due on April 5, 2026. The team had six people in it, and the work was divided between each and every person so that not one person had to do all the work. That was one of the reasons we could meet the deadlines.

ii. Also provide the details for:

1. Whether weekends will be counted in your schedule or not

weekends are counted in the project schedule because the team did not necessarily spend the whole day working on the project on weekends. Weekends were still used for team meetings reviewing the progress and making some changes or final revisions for the submission.

2. What is the number of working hours per day for the project?

The estimated working time per day for the project for all six people would be around two hours on average. This is a reasonable estimate as the work was divided between six people, and each person had to do their own thing, so instead of spending the whole day, a bit of time during the whole day was okay for this project.

Project Cost:

We used the application composition method, using an article from GeeksForGeeks [1] as a reference, to estimate the cost, effort, and pricing. First, we estimate the number of screens, reports, and 3GL components, and then calculate their object points based on the complexity weight table below:

Object Type	Complexity Weight		
	Simple	Medium	Difficult

Screen	1	2	3
Report	2	5	8
3GL Components	-	-	10

For our project, the number of objects for each type given the features planned is estimated as:

Object Type	Estimation Count		
	Simple	Medium	Difficult
Screen	0	5	2
Report	2	2	0
3GL Components	0	0	6

This is a total of 90 object points. Since we will reuse components like the search engine, reader, etc., which is about 35% reuse. After adjusting, we compute a new object point of:

$$NOP = 90 * (1 - 0.35) = 58.5$$

Our team comprises relatively low-experience developers, estimated to have a productivity (PROD) of 6 each. Therefore, our effort is estimated to be:

$$Effort = NOP/PROD = 58.5/6 = 9.75 \text{ person} - \text{months}$$

Therefore, with 5 developers, it will take approximately 2 months to develop the program.

Hardware Cost: \$15/month

- AWS Cloud Server: \$15/month
- Development/testing Computers (Preowned): \$0

Software Cost: \$300

- IDE: \$0
- Libraries and Frameworks: \$0
- Automated Testing Tool: \$100
- UI/UX Tools: \$200

Personnel Cost: \$1000 development + \$40/month maintenance cost

- Developers (5): \$100/month each for 2 months
- Training for maintenance (2): \$40/month

Total Cost: \$1300 fixed + \$55/month service cost after development**Price: \$2000 for development, then \$70/month of live service****Comparison with Similar Designs**

The proposed bookshelf application belongs to the same general domain as digital reading platforms such as Amazon Kindle, Google Play Books, and Apple Books. These systems already support important reader functions such as opening digital books, navigating pages, and bookmarking content. However, prior usability research on eBook readers reports frequent issues with navigation, interface complexity, and document management, which suggests that there is still room for simpler and more user-focused designs [2].

System	Strengths	Limitations	Why Bookshelf is different
Amazon Kindle	Strong reading experience, bookmarking, supports personal documents through send to Kindle [3].	Personal files are typically routed through Amazon services, so local file handling feels less direct for users who want a lightweight, download-folder based workflow.	Our Bookshelf emphasizes direct import from the device Downloads folder and a simpler interface for locally managed reading.
Google Play Books	Supports uploading	Best fit for users	Bookshelf focuses on

	PDF and EPUB files and can sync reading tools across devices [4].	already working inside a Google account ecosystem, interface and sync features may be more than a lightweight local reader needs.	core reading features without depending on a large cloud-centered workflow.
Apple Books	Provides built-in reading and bookmarking features for users in the Apple ecosystem [5].	Most convenient for Apple users and less ideal as a cross-platform, design reference for a simple, platform independent reader.	Bookshelf is designed around portability of the design itself, simple file import, basic navigation, and offline friendly reading
Bookshelf (proposed)	Direct local import .txt and .pdf support, simple navigation, bookmaking, and a lightweight reader flow.	Doesn't aim to compete with the size of commercial content stores or advanced cloud ecosystem features.	Targets a narrower but more clear goal, easy management of personal book files with a clean reading experience.

Compared with similar designs, our bookshelf application is not trying to replace large commercial ecosystems. Instead, it addresses a narrower need, which is a lightweight reading application that gives users direct control over locally stored files while still supporting essential features such as page navigation and bookmarking. This design choice is consistent with our project's emphasis on simplicity, usability, and efficient file management.

Conclusion:

Changes & Deviations:

1. We refined our functional requirements to be more atomic, ensuring each requirement represents a single, independent, and testable behavior.
2. We adjusted our software reuse target to 35%, rather than developing all components from scratch.

Justification:

1. These changes were justified to improve the testability and maintainability of the system. Improved atomicity allowed a more rigorous test plan with clear "True/False" execution cases. This ensures for more effective unit testing as demonstrated in our project test results.

2. The 35% software reuse strategy was essential to maintain a stable profit margin and adhere the 10-week development timeline. We ensured the project's economic feasibility by reducing the New Object Points (NOP) to 58.5.

References

- [1] GeeksforGeeks, "Application composition estimation model (COCOMO II: Stage 1) - software engineering," GeeksforGeeks, <https://www.geeksforgeeks.org/software-engineering/software-engineering-application-composition-estimation-model-cocomo-ii-stage-1/> (accessed Apr. 2, 2026).
- [2] J. V. Richardson Jr. and K. Mahmood, "eBook Readers: User Satisfaction and Usability Issues," *Library Hi Tech*, vol. 30, no. 1, pp. 170–185, 2012.
- [3] Amazon, "Learn About Sending Documents to Your Kindle Library," Amazon Help, accessed Apr. 3, 2026.
- [4] Google, "Upload PDF and EPUB files to your library," Google Play Books Help, accessed Apr. 3, 2026.
- [5] Apple, "Read books in the Books app on iPhone / Bookmark a page," Apple Support, accessed Apr. 3, 2026.